

# LAB 14 – Creating a Pipeline Job and Triggering Builds Using GitHub Webhooks

In this lab, you will create a **Pipeline Job** that reads its pipeline definition (`Jenkinsfile`) directly from a GitHub repository. You will also configure a **GitHub Webhook** so that every code change automatically triggers a new Jenkins build.

---

## Prerequisites

Before starting this lab, ensure the following:

- Jenkins is installed and running.
  - Git is installed on the Jenkins server.
  - Docker is installed on the Jenkins server.
  - **Git** and **Docker** plugins are installed in Jenkins.
  - The Jenkins user has permission to execute Docker commands without using `sudo`.
- 

## Step 1 – Fork the Repository

Fork the following repository into your own GitHub account:

```
https://github.com/ajit010/ci-cd-pipeline.git
```

After forking, copy your repository URL.

Example:

```
https://github.com/<your-github-username>/ci-cd-pipeline.git
```

---

## Step 2 – Give Jenkins Access to Docker

By default, the Jenkins user cannot execute Docker commands.

Add the Jenkins user to the Docker group:

```
sudo usermod -aG docker jenkins
```

If this is the first time adding the user to the Docker group, refresh the group membership:

```
newgrp docker
```

### Note

If you already configured Docker permissions in a previous lab, you can skip the `newgrp docker` command.

Restart Jenkins:

```
sudo systemctl restart jenkins
```

Verify that the Jenkins user can execute Docker commands:

```
sudo su - jenkins
```

```
docker ps
```

The command should execute without requiring sudo.

---

## Step 3 – Create a Pipeline Job

From the Jenkins Dashboard:

```
New Item
```

Enter a job name.

Example:

```
pipeline-demo
```

Select:

```
Pipeline
```

Click **OK**.

---

## Step 4 – Configure Pipeline from SCM

Scroll to the **Pipeline** section.

Select:

```
Definition
```

```
↓
```

```
Pipeline script from SCM
```

SCM:

```
Git
```

Repository URL:

```
https://github.com/<your-github-username>/ci-cd-pipeline.git
```

Branch:

```
*/main
```

or

```
*/master
```

depending on your repository.

Leave the **Script Path** as:

```
Jenkinsfile
```

Jenkins will automatically read the `Jenkinsfile` from the repository during every build.

Save the job.

---

## Step 5 – Enable GitHub Webhook Trigger

Open the job configuration.

Under **Build Triggers**, enable:

```
GitHub hook trigger for GITScm polling
```

Save the job.

---

## Step 6 – Configure the GitHub Webhook

Open your GitHub repository.

Navigate to:

```
Settings
```

```
↓
```

```
Webhooks
```

```
↓
```

```
Add Webhook
```

Payload URL:

```
http://<YOUR-PUBLIC-IP>:8080/github-webhook/
```

Example:

```
http://18.234.36.131:8080/github-webhook/
```

**Replace the IP address with your own Jenkins server's public IP address.**

Content Type:

```
application/json
```

Events:

```
Just the push event
```

Click:

Add Webhook

---

## Step 7 – Test the Webhook

Modify any file in your repository.

Example:

index.html

Commit and push the changes.

Within a few seconds, GitHub sends a webhook request to Jenkins.

Jenkins automatically starts a new build without clicking **Build Now**.

You should see a new build appearing in the Build History.

---

## Expected Result

- Code is pushed to GitHub.
- GitHub sends a webhook request.
- Jenkins receives the webhook.
- Jenkins pulls the latest code.
- The Pipeline executes automatically.

This is the foundation of a CI/CD workflow.

---

## LAB 15 – Configuring Jenkins Agent Mode

In this lab, you will configure Jenkins to execute builds on a separate machine called an **Agent Node**.

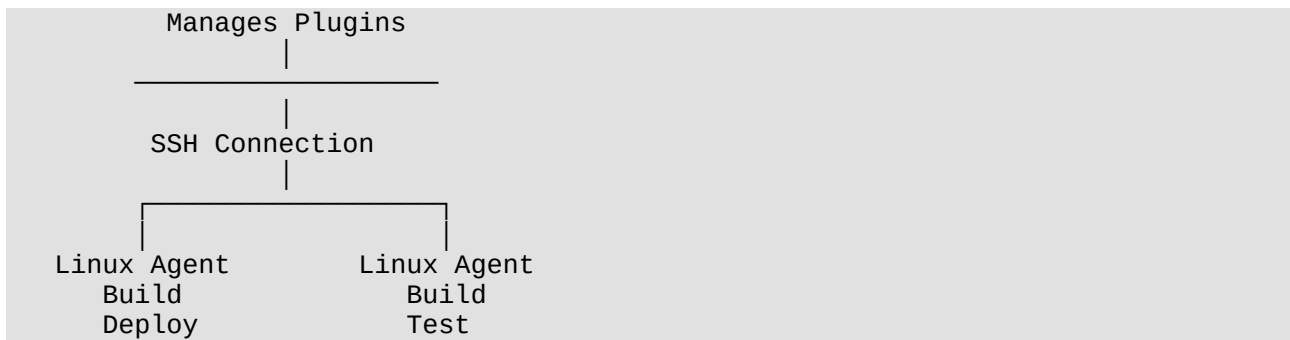
Instead of running all builds on the Jenkins Controller, the Controller delegates build execution to one or more Agents.

---

## Jenkins Architecture

Jenkins Controller  
(Master / Controller)

Stores Jobs  
Hosts Jenkins UI  
Schedules Builds



---

## Controller (Master) Responsibilities

The Controller is responsible for:

- Hosting the Jenkins Web UI
- Managing plugins
- Storing job configurations
- Scheduling builds
- Maintaining build history
- Assigning work to available agents

Recommended setup:

- Ubuntu 24.04
- OpenJDK 17
- Jenkins installed

---

## Agent Responsibilities

An Agent is a worker machine that performs the actual work.

Typical tasks include:

- Building applications
- Running tests
- Creating Docker images
- Deploying applications

Recommended setup:

- Ubuntu 24.04
- OpenJDK 17
- SSH enabled

- A dedicated jenkins user
- 

## Part 1 – Prepare the Agent Machine

Launch a new EC2 instance.

Recommended configuration:

- Ubuntu 24.04
- t3.micro
- 8 GB SSD

When launching the instance:

- Select **Proceed without a Key Pair**.

In the **Advanced Details** section, paste the following **User Data** script.

```
#cloud-config

ssh_pwauth: true

chpasswd:
  expire: false
  users:
    - name: ubuntu
      password: ajit@1234
      type: text

runcmd:
  - sed -i 's/^#\?PasswordAuthentication.*/PasswordAuthentication yes/'
    /etc/ssh/sshd_config
  - sed -i 's/^#\?KbdInteractiveAuthentication.*/KbdInteractiveAuthentication
yes/' /etc/ssh/sshd_config
  - systemctl restart ssh
```

This enables password-based SSH authentication for the Ubuntu user.

---

## Step 2 – Connect to the Agent

From your local machine:

```
ssh ubuntu@<PUBLIC-IP>
```

Enter the password configured in the cloud-init script.

---

## Step 3 – Install Java

Update the package list:

```
sudo apt update -y
```

Install Java:

```
sudo apt install openjdk-17-jdk -y
```

Verify the installation:

```
java --version
```

---

## Step 4 – Create the Jenkins User

Create a dedicated Jenkins user:

```
sudo useradd -m -s /bin/bash jenkins
```

Set a password:

```
sudo passwd jenkins
```

Enter a password when prompted.

### Important

Remember this password. It will be used when configuring the Jenkins Agent.

Give the Jenkins user sudo access:

```
sudo usermod -aG sudo jenkins
```

---

## Verify SSH Service

Check whether SSH is running:

```
sudo systemctl status ssh
```

Expected status:

```
active (running)
```

---

## Verify SSH Connectivity

From the Jenkins Controller machine, test SSH connectivity:

```
ssh jenkins@<AGENT-IP>
```

If prompted for a password, enter the password you created earlier.

If the login succeeds, SSH connectivity is working correctly.

---

# Generate SSH Keys on the Jenkins Controller

Log in to the Jenkins Controller.

Generate a new SSH key pair:

```
ssh-keygen -t rsa -b 4096
```

Press **Enter** for all prompts to accept the default values.

The keys are stored in:

```
~/.ssh/id_rsa
```

and

```
~/.ssh/id_rsa.pub
```

---

## Copy the Public Key to the Agent

Run:

```
ssh-copy-id jenkins@<AGENT-IP>
```

Enter the Jenkins user's password one final time.

This copies the public key to the agent.

---

## Test Passwordless Login

Now test SSH again:

```
ssh jenkins@<AGENT-IP>
```

This time, SSH should log in **without asking for a password**.

---

## Add SSH Credentials in Jenkins

Open Jenkins.

Navigate to:

```
Manage Jenkins
```

```
↓
```

```
Credentials
```

Click:

```
Add Credentials
```



Choose:

Kind:

SSH Username with private key

Username:

jenkins

Private Key:

Copy the contents of:

```
cat ~/.ssh/id_rsa
```

Paste the complete private key into Jenkins.

Save the credential.

---

## Create a Jenkins Agent

Navigate to:

Manage Jenkins

↓

Nodes

↓

New Node

Agent Name:

linux-agent-demo

Type:

Permanent Agent

Click **Create**.

---

## Configure the Agent

Remote Root Directory:

/home/jenkins

This directory must already exist on the agent machine.

Labels:

linux-agent

(You may choose any label.)

Usage:

Use this node as much as possible

Launch Method:

Launch agents via SSH

Host:

<AGENT-IP>

Credentials:

Select the SSH credentials created earlier.

Host Key Verification Strategy:

For lab environments:

Non-verifying Verification Strategy

### Production Recommendation

Use **Known Hosts Verification Strategy** instead of the non-verifying option for better security.

Click **Save**.

---

## Start the Agent

Open the Agent page.

Click:

Launch Agent

Jenkins connects to the remote machine over SSH.

If everything is configured correctly, the Agent status changes to:

Online

---

## Expected Result

You have successfully configured a Jenkins Agent.

The Jenkins Controller can now:

- Connect to the agent over SSH
- Execute builds remotely

- Distribute workloads across multiple machines
- Scale Jenkins by adding additional agents as needed

This architecture is commonly used in production environments to improve scalability, performance, and resource utilization.

---